

SAT AUTO

Système de Gestion d'Auto-École

DOCUMENT DE GESTION DES INCIDENTS

Problèmes rencontrés et solutions apportées

Version 1.0 Avril 2026 | BTS SIO SLAM

1. Introduction

Ce document recense les principaux incidents et problèmes techniques rencontrés au cours du développement du projet SAT AUTO. Pour chaque incident, on y trouve : la description du problème, la cause identifiée, la solution apportée et les compétences acquises.

Ce document fait partie intégrante de la documentation du PPE BTS SIO SLAM et atteste de la démarche de résolution de problèmes adoptée tout au long du projet.

Légende des niveaux de gravité

Niveau	Description
CRITIQUE	Bloquant, l'application ne fonctionne pas du tout
MAJEUR	Impact fonctionnel important, une fonctionnalité principale ne marche pas
MINEUR	Gêne limitée, fonctionnalité secondaire ou problème d'affichage

2. Synthèse des Incidents

#	Catégorie	Titre	Gravité	Statut
1	Base de données	Erreur PDO : connexion échouée	CRITIQUE	Résolu
2	Architecture MVC	Inclusion de fichiers introuvables	CRITIQUE	Résolu
3	Authentification	Boucle infinie de redirection	CRITIQUE	Résolu
4	SQL / BDD	Doublon inscription cours théorique	MAJEUR	Résolu
5	PHP / Sessions	Données de session perdues après déconnexion	MAJEUR	Résolu
6	CSS / Affichage	Style cassé sur le tableau de bord admin	MINEUR	Résolu
7	PHP / Forms	Données POST perdues après redirection	MAJEUR	Résolu
8	SQL / Contraintes	Erreur de suppression (contrainte FK)	MAJEUR	Résolu
9	PHP / Logique	Mauvaise redirection selon le rôle	MAJEUR	Résolu
10	Sécurité	Accès à une page admin sans être connecté	CRITIQUE	Résolu

3. Fiches Détaillées des Incidents

INCIDENT #1 – Erreur PDO : connexion à la base de données échouée

Catégorie	Base de données	Gravité	CRITIQUE
-----------	-----------------	---------	----------

Description du problème

Au lancement de l'application, une erreur fatale PHP apparaissait immédiatement : « SQLSTATE[HY000] [1049] Unknown database 'sat_auto' ». Aucune page ne s'affichait.

Cause identifiée

La base de données sat_auto n'avait pas encore été créée dans MySQL. Le fichier SQL n'avait pas été importé avant de tenter d'accéder à l'application. De plus, la configuration PDO dans modele_class.php ne gérait pas l'exception de connexion avec un message clair.

Solution apportée

- Import du fichier SQL/sat_auto.sql via phpMyAdmin pour créer la base
- Ajout d'un bloc try/catch sur la connexion PDO dans modele_class.php
- Affichage d'un message d'erreur clair en cas d'échec de connexion
- Vérification de la correspondance host / user / password dans le fichier de config

Ce que cela m'a appris

L'importance de toujours vérifier l'état de la base de données avant de tester l'application. J'ai aussi appris à gérer correctement les exceptions PDO pour obtenir des messages d'erreur exploitables.

INCIDENT #2 – Inclusion de fichiers introuvables (require_once)

Catégorie	Architecture MVC	Gravité	CRITIQUE
-----------	------------------	---------	----------

Description du problème

Une erreur PHP fatale s'affichait : « Failed opening required 'controleur.class.php' ». L'application ne pouvait pas charger les classes nécessaires.

Cause identifiée

Les chemins d'inclusion dans les require_once étaient relatifs et incorrects après la réorganisation des fichiers dans des sous-dossiers (controleur/, modele/, vue/). Par exemple, gestion_admin.php appelait require_once('controleur.class.php') alors qu'il se trouvait dans le même dossier.

Solution apportée

- Correction de tous les chemins avec des chemins relatifs corrects (require_once('../modele/modele_class.php'))
- Mise en place d'une convention de nommage et de structure de dossiers claire dès le début
- Test systématique après chaque déplacement de fichier

Ce que cela m'a appris

J'ai compris l'importance de définir l'arborescence du projet avant de commencer le développement. Les chemins d'inclusion doivent être planifiés et testés immédiatement.

INCIDENT #3 – Boucle infinie de redirection après connexion

Catégorie	Authentification	Gravité	CRITIQUE
-----------	------------------	---------	----------

Description du problème

Après une connexion réussie, le navigateur affichait une erreur « ERR_TOO_MANY_REDIRECTS ». L'utilisateur était bloqué en boucle et ne pouvait jamais accéder à son tableau de bord.

Cause identifiée

Dans index.php, la condition de redirection après connexion vérifiait si l'utilisateur était connecté ET si la page était 'accueil', puis redirigeait vers le dashboard. Mais le dashboard redirigeait aussi vers 'accueil' dans certains cas, créant une boucle. Le rôle en session n'était pas correctement lu.

Solution apportée

- Révision complète de la logique de redirection dans index.php
- Vérification que \$_SESSION['role'] est bien défini immédiatement après connecterUtilisateur()
- Ajout de exit() systématique après chaque header('Location:')
- Test de chaque rôle individuellement (admin, client, moniteur)

Ce que cela m'a appris

Toujours placer exit() après un header('Location:') en PHP. J'ai aussi appris à bien tracer la valeur des variables de session avec var_dump() pour déboguer les problèmes de redirection.

INCIDENT #4 – Doublet lors de l'inscription à un cours théorique

Catégorie	SQL / Base de données	Gravité	MAJEUR
-----------	-----------------------	---------	--------

Description du problème

Un client pouvait s'inscrire plusieurs fois au même cours théorique, créant des doublons dans la table inscription_theorique et faussant le comptage des places disponibles.

Cause identifiée

La requête INSERT INTO inscription_theorique ne vérifiait pas préalablement si le client était déjà inscrit au cours ciblé. Il manquait une contrainte d'unicité et une vérification côté PHP.

Solution apportée

- Ajout d'une contrainte UNIQUE KEY unique_inscription (id_cours, id_client) dans le schéma SQL
- Ajout d'une vérification PHP avant l'INSERT : SELECT COUNT(*) pour détecter le doublon
- Retour d'un message d'erreur explicite : « Vous êtes déjà inscrit à ce cours »

Ce que cela m'a appris

L'importance des contraintes d'intégrité au niveau de la base de données pour éviter les anomalies, même si la logique PHP est censée prévenir le problème. La BDD est le dernier rempart.

INCIDENT #5 – Données de session incorrectes après déconnexion partielle

Catégorie	PHP / Sessions	Gravité	MAJEUR
-----------	----------------	---------	--------

Description du problème

Après une déconnexion, certains utilisateurs pouvaient encore accéder à des pages protégées en utilisant le bouton retour du navigateur. Les données de session semblaient persistantes.

Cause identifiée

La fonction de déconnexion n'appelait que `session_unset()` mais pas `session_destroy()`. De plus, il n'y avait pas de vérification de session au début des pages protégées dans certains cas.

Solution apportée

- Correction de la méthode `deconnecterUtilisateur()` : `session_unset()` + `session_destroy()`
- Ajout d'un header no-cache pour empêcher le cache navigateur de restaurer des pages protégées
- Vérification systématique de `estConnecte()` et du rôle au début de chaque case du routeur

Ce que cela m'a appris

La déconnexion PHP doit être complète : `session_unset()` vide les variables, `session_destroy()` tue la session. Les deux sont nécessaires.

INCIDENT #6 – Mise en page cassée sur le tableau de bord administrateur

Catégorie	CSS / Affichage	Gravité	MINEUR
-----------	-----------------	---------	--------

Description du problème

Certains éléments du tableau de bord admin se superposaient ou débordaient de leur conteneur sur certaines résolutions d'écran, rendant la lecture des tableaux difficile.

Cause identifiée

Les largeurs des colonnes de tableaux HTML étaient définies en pixels fixes sans tenir compte de la largeur totale du conteneur parent. Certains flex-containers manquaient de propriétés `overflow`.

Solution apportée

- Remplacement des largeurs fixes par des largeurs en pourcentage sur les tableaux
- Ajout de `overflow-x: auto` sur les conteneurs de tableaux pour permettre le scroll horizontal
- Révision des media queries dans `style.css`
- Tests sur différentes résolutions (1920px, 1366px, 1280px)

Ce que cela m'a appris

En CSS, il vaut mieux utiliser des unités relatives (% , fr , em) plutôt que des valeurs fixes en pixels pour garantir la robustesse de la mise en page sur différentes tailles d'écran.

INCIDENT #7 – Données de formulaire perdues après redirection POST

Catégorie	PHP / Formulaires	Gravité	MAJEUR
-----------	-------------------	---------	--------

Description du problème

Après la soumission d'un formulaire (ajout client, ajout moniteur...), la page se rechargeait mais les messages de succès ou d'erreur n'apparaissaient pas toujours correctement. Parfois les données semblaient disparaître.

Cause identifiée

L'application utilisait le pattern redirect-after-POST mais les messages flash n'étaient pas correctement transmis via la session. Certains cas ne stockaient pas le message avant la redirection.

Solution apportée

- Standardisation de l'utilisation de `$_SESSION['success']` et `$_SESSION['error']` pour tous les messages
- Affichage systématique et suppression (unset) de ces variables en début de vue
- Vérification que chaque action POST se termine toujours par une redirection `header()`

Ce que cela m'a appris

Le pattern PRG (Post/Redirect/Get) est essentiel en PHP pour éviter la double soumission et gérer proprement les messages de retour via les variables de session.

INCIDENT #8 – Erreur SQL lors de la suppression d'un client ou moniteur

Catégorie	SQL / Contraintes	Gravité	MAJEUR
-----------	-------------------	---------	--------

Description du problème

En essayant de supprimer un client qui avait des cours pratiques ou des examens associés, MySQL retournait une erreur : « Cannot delete or update a parent row: a foreign key constraint fails ».

Cause identifiée

Les clés étrangères des tables `cours_pratique` et `examen` référencent la table `client` avec une contrainte d'intégrité. La suppression en cascade (`ON DELETE CASCADE`) n'avait pas été définie lors de la création initiale des tables.

Solution apportée

- Ajout de `ON DELETE CASCADE` sur toutes les clés étrangères dans le script SQL
- Régénération du script `sat_auto.sql` avec les bonnes contraintes
- Test de suppression sur des clients ayant des données liées pour valider le comportement

Ce que cela m'a appris

Les contraintes de clés étrangères sont importantes pour l'intégrité des données, mais elles doivent être pensées dès la conception : faut-il une suppression en cascade ou une restriction avec message d'erreur métier ?

INCIDENT #9 – Mauvaise redirection après connexion selon le rôle

Catégorie	PHP / Logique métier	Gravité	MAJEUR
-----------	----------------------	---------	--------

Description du problème

Un moniteur connecté était parfois redirigé vers le tableau de bord client, ou inversement. Le routage post-connexion ne respectait pas systématiquement le rôle de l'utilisateur.

Cause identifiée

La méthode `redirigerSelonRole()` lisait `$_SESSION['role']` mais cette variable n'était pas encore définie au moment de l'appel dans certains cas de figure. L'ordre des opérations dans `connecterUtilisateur()` était incorrect.

Solution apportée

- Révision de `connecterUtilisateur()` pour définir `$_SESSION['role']` et `$_SESSION['user_id']` avant tout retour
- Ajout de tests unitaires manuels pour chaque rôle
- Ajout de logs temporaires (`error_log`) pour tracer la valeur du rôle pendant le débogage

Ce que cela m'a appris

L'ordre d'exécution en PHP est critique. Il faut toujours s'assurer que les variables sont correctement définies avant d'en dépendre. Les logs d'erreur PHP sont un outil précieux pour déboguer.

INCIDENT #10 – Accès à une page protégée sans être authentifié

Catégorie	Sécurité	Gravité	CRITIQUE
-----------	----------	---------	----------

Description du problème

En entrant directement l'URL `?page=dashboard_admin` dans le navigateur, il était possible d'accéder partiellement au contenu de la page admin sans être connecté, ou d'obtenir des erreurs PHP exposant la structure du code.

Cause identifiée

La vérification d'authentification n'était pas en place sur toutes les routes sensibles au démarrage du projet. Certaines pages chargeaient des données avant de vérifier le rôle de l'utilisateur.

Solution apportée

- Ajout d'une vérification `estAdmin()` / `estClient()` / `estMoniteur()` en tout premier dans chaque case du switch
- Redirection immédiate vers `?page=connexion` si la vérification échoue
- Masquage des messages d'erreur PHP en production (`display_errors = Off`)

Ce que cela m'a appris

Le contrôle d'accès doit être la première instruction de chaque route protégée, pas la dernière. La sécurité par obscurité (cacher les erreurs) est nécessaire mais insuffisante : il faut des contrôles explicites.

4. Bilan et Compétences Acquises

La résolution de ces 10 incidents a permis de renforcer significativement les compétences techniques et méthodologiques suivantes :

Domaine	Compétences renforcées
PHP / Back-end	Gestion des sessions, pattern PRG, require_once, gestion des exceptions
Base de données	Contraintes FK, CASCADE DELETE, contraintes UNIQUE, débogage PDO
Architecture MVC	Organisation des dossiers, chemins d'inclusion, séparation des responsabilités
Sécurité	Contrôle d'accès par rôle, gestion des sessions, masquage des erreurs
CSS / Front	Responsive design, unités relatives, overflow, flexbox
Méthode	Débogage par logs, tests systématiques, traçabilité des incidents

Ce document d'incidents témoigne de la démarche professionnelle adoptée tout au long du projet SAT AUTO : chaque problème rencontré a fait l'objet d'une analyse, d'une résolution documentée et d'une capitalisation d'expérience.

Document rédigé dans le cadre du PPE BTS SIO SLAM – SAT AUTO – Version 1.0 – Avril 2026.